DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE



CONCURRENT AND REAL TIME PROGRAMMING

[INQ0091623] AA 2026-27

Lab 6

Process Synchronization

Andrea Rigoni Garola <andrea.rigonigarola@unipd.it>

Synchronization of processes using IPC

In computer science, inter-process communication (IPC) refers to the set of mechanisms that an operating system provides to allow the user processes to manage **shared data**.

There can be several methods to communicate among processes:

- sockets (we will see them)
- pipes (standard IO)
- signals
- message queues
- semaphores
- shared memory

We will see also see that threads can be locked:

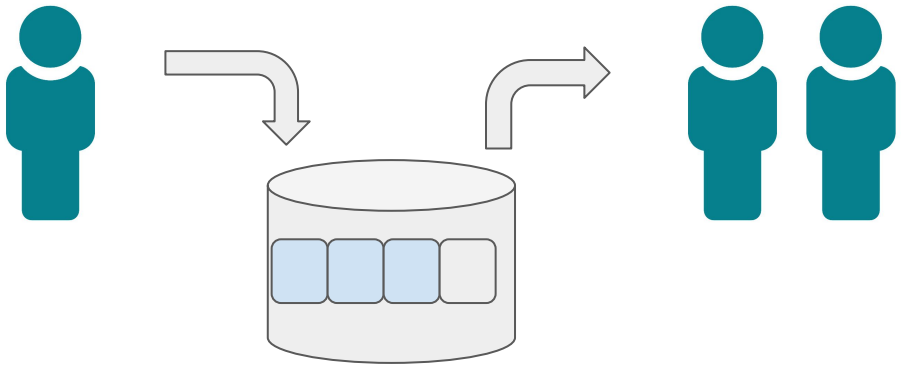
- mutexes and condition variables
-

Here we are focusing on which of them can help us to synchronize our processes during the concurrent execution.

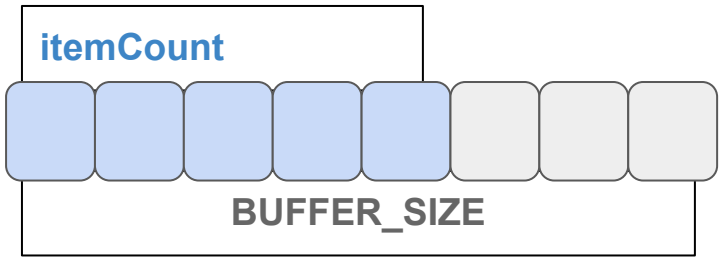
The producer-consumer problem

A typical example of a multi-process synchronization problem

The **producer** repeatedly generates data and writes it into the buffer. At the same time the **consumer** independently reads the data in the buffer, removing it in the course of reading it, and using that data in some way.



Example 1 - Sleep and wakeup with a counter



Process 1

```
int itemCount = 0;

void producer() {
   int item = 0;
   while (1) {
        item = produceItem();
        if (itemCount == BUFFER_SIZE) { sleep(); }
        putItemIntoBuffer(item);
        itemCount = itemCount + 1;
        if (itemCount == 1) { wakeup(consumer); }
   }
}
```

Process 2

```
void consumer() {
   int item;
   while (1) {
        if (itemCount == 0) { sleep(); }
        item = removeItemFromBuffer();
        itemCount = itemCount - 1;
        if (itemCount == BUFFER_SIZE - 1) { wakeup(producer); }
        consumeItem(item);
   }
}
```

Hazard 1 - Lost wakeup → deadlock (race condition)

at start **itemCount = 0**

Process 1

```
void producer() {
    int item = 0;
    while (1) {
        item = produceItem();
        if (itemCount == BUFFER_SIZE) { sleep(); }
        putItemIntoBuffer(item);
        itemCount = itemCount + 1;
        if (itemCount == 1) {
            wakeup(consumer); }
    }
}
```

context is switched before the consumer goes to sleep

context switch

context switch

consumer was not sleeping yet so it misses the wakeup

Process 2

```
void consumer() {
    int item;
    while (1) {
        if (itemCount == 0) {
                sleep(); }
        item = removeItemFromBuffer();
        itemCount = itemCount - 1;
        if (itemCount == BUFFER_SIZE - 1) {
                wakeup(producer); }
        consumeItem(item);
    }
}
```

Race condition: the test `itemCount == 0` and `sleep()` are not one atomic step. The producer later fills the buffer and sleeps too: both sleep forever. Also `itemCount = itemCount ± 1` is itself a data race (Hazard 6).

Example 2 - Circular (ring) buffer

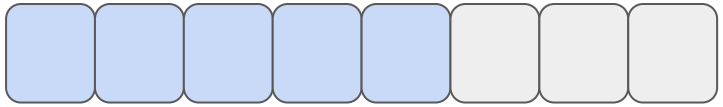
CODING EXAMPLE

```
struct BufferData {
 int readIdx;
 int writeIdx;
 int buffer[BUFFER_SIZE];
};

int next(int id) { return (id+1)%BUFFER_SIZE; }

/* producer routine */
static void producer() {
 int item = 0;
 while(1) {
   if(next(sharedBuf->writeIdx) != sharedBuf->readIdx) {
   /* Write data item */
   sharedBuf->buffer[sharedBuf->writeIdx] = item;
   /* Update write index */
   sharedBuf->writeIdx = next(sharedBuf->writeIdx);
 }}
}

/* Consumer routine */
static void consumer() {
 int item;
 while(1) {
  if(sharedBuf->readIdx != sharedBuf->writeIdx) {
   /* Get data item */
   item = sharedBuf->buffer[sharedBuf->readIdx];
   /* Update read index */
   sharedBuf->readIdx = next(sharedBuf->readIdx);
 }}
}
```



Single producer, single consumer (SPSC)

`readIdx`: next slot to read, written only by the consumer

`writeIdx`: next free slot, written only by the producer

empty: `readIdx == writeIdx`

full: `next(writeIdx) == readIdx` (one slot is always left free to tell full from empty)

The order of the operations is right: write the data, then publish the index; read the data, then free the slot.

But it is still broken: plain `int` shared between two processes is a data race. The next slides show every way it can fail.

Race condition vs data race

Race condition

a property of the program logic

The result **depends on the timing** (the interleaving) of the operations.

Typical shape: a **check** and the **action** that depends on it are not one atomic step: check-then-act, read-modify-write.

```
if (itemCount == 0)
    sleep();   /* lost wakeup */
```

Data race

a property of memory accesses

Two threads/processes access the **same memory location** at the same time, **at least one writes**, and nothing orders the accesses (no lock, no atomic).

In C11/C++11 this is **undefined behaviour**: compiler and CPU may reorder, cache in registers or split the accesses.

```
/* producer */          /* consumer */
writeIdx = next(w);     if (readIdx != writeIdx)
```

They are independent: fixing one does not fix the other

Data race, no race condition

Example 2 with 1 producer + 1 consumer: the logic is right, but plain `int` accesses can be reordered or torn (Hazards 3, 4).

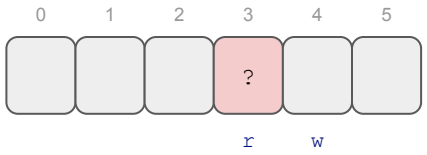
Race condition, no data race

Make every access atomic and add a second producer: `load(writeIdx)` … `store(writeIdx+1)` is still a check-then-act (Hazard 5).

Hazard 2 - Publish before write (race condition)

Same SPSC buffer, but the producer updates `writeIdx` before writing the slot. At start: `readIdx = writeIdx = 3`

Process 1	Process 2
t1 `w = writeIdx;  /* w = 3 */`	
t2 `if (next(w) != readIdx) {  /* room */`	
t3 `writeIdx = next(w);  /* publish 4 */`	
context switch	
	t4 `if (readIdx != writeIdx) {  /* 3 != 4 */`
	t5 `item = buffer[3];  /* NOT written yet */`
	t6 `readIdx = next(3);  /* 4 */`
context switch	
t7 `buffer[w] = item;  /* too late */`	



Effect

The consumer reads a **stale / empty slot** (whatever was left from the previous lap).

The real item is written after the slot was consumed: it is **never read**.

Mirror bug on the consumer side: advancing `readIdx` before copying `buffer[readIdx]` frees the slot too early, so the producer can **overwrite an unread slot**.

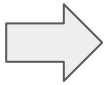
Rule: write the data, then publish the index. Read the data, then release the slot.

Hazard 3 - Reordering and caching (data race)

The source of Example 2 is in the right order, yet with plain `int` there is no guarantee the other process sees it in that order.

What you wrote

```
buffer[writeIdx] = item;       /* (1) */
writeIdx = next(writeIdx);     /* (2) */
```

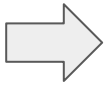


What may become visible

```
writeIdx = next(writeIdx);     /* (2) seen first */
buffer[w] = item;              /* (1) seen later */
```

The compiler may swap two independent stores, and the CPU store buffer (ARM, POWER, RISC-V) may make (2) visible to the other core before (1). The consumer then reads the slot early: **exactly Hazard 2, with no bug in the source**.

```
/* consumer busy-waits for data */
while (readIdx == writeIdx) ;
```



```
int w = writeIdx;      /* loaded once */
while (readIdx == w) ;   /* forever */
```

Nobody else writes `writeIdx` in this loop as far as the compiler knows, so it may keep it in a register: **the consumer never sees new data**.

Why `volatile` is not the fix: it stops the register caching, but it does not stop the CPU reordering and does not create any ordering between the data and the index.

Fix: C11 atomics with `memory_order_release` on the index store and `memory_order_acquire` on the index load, or a mutex / semaphore, which contain the needed barriers.

Hazard 4 - Torn index updates (data race)

A. Index wider than the machine word

```
uint64_t writeIdx;  /* free-running counter */
/* on a 32-bit CPU, writeIdx++ is: */
store low  32 bits
store high 32 bits
```

Going from `0x0_FFFFFFFF` to `0x1_00000000`, a reader between the two stores sees `0x0_00000000`: a value that **never existed**.

The empty/full test is wrong and the slot number is garbage: the consumer reads **empty slots** or the producer **overwrites unread ones**.

B. Indices packed in one word

```
struct { unsigned readIdx  : 16;
         unsigned writeIdx : 16; } idx;
/* producer */
idx.writeIdx = next(idx.writeIdx);
```

A bit-field store is **load word, change 16 bits, store word**.

If the consumer updates `readIdx` between the producer's load and store, the producer writes back the old `readIdx`: a **lost update** on a field it never meant to touch. The slot is read twice later.

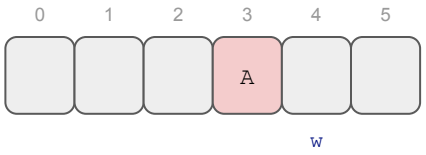
Fix: each index is its own naturally aligned atomic object (`_Atomic unsigned`), never a bit-field or a split value. Check `atomic_is_lock_free()` : in shared memory between processes a lock-based atomic does not work.

Bonus: put the two indices on different cache lines to avoid false sharing.

Hazard 5 - Two producers: lost update, overwrite

Each index now has **two writers**. Even with atomic loads and stores this is a race condition. At start `writeIdx = 3`

Producer 1	Producer 2
t1 `w = writeIdx;` `/* w = 3 */`	
t2 `if (next(w) != readIdx) {` `/* room */`	
context switch	
	t3 `w = writeIdx;` `/* w = 3 */`
	t4 `buffer[3] = B;`
	t5 `writeIdx = next(w);` `/* 4 */`
context switch	
t6 `buffer[3] = A;` `/* overwrites B */`	
t7 `writeIdx = next(w);` `/* 4 again */`	



Effect

Two items produced, one stored: B is **overwritten** (the consumer may even be reading it).

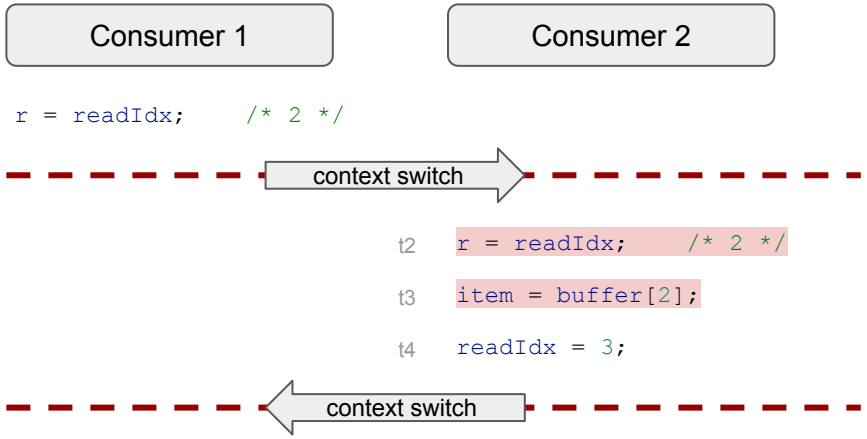
Two increments, one kept: **lost update** of `writeIdx`.

Near full: both producers pass the room test for the last free slot; if one re-reads `writeIdx` it writes past the limit and **overwrites an unread slot**.

Fix: the check, the write and the index update must be one critical section (mutex), or reserve the slot atomically.

Hazard 6 - Two consumers and shared counters

Two consumers, `readIdx = 2`



The same item is **consumed twice** and one increment of `readIdx` is **lost**.

`itemCount = itemCount + 1` (Example 1)

who	instruction	itemCount
Producer	`LOAD R1 ← itemCount`	4
Consumer	`LOAD R2 ← itemCount`	4
Consumer	`SUB R2 ← R2 - 1`	4
Consumer	`STORE itemCount ← R2`	3
Producer	`ADD R1 ← R1 + 1`	3
Producer	`STORE itemCount ← R1`	5

One + and one − should leave 4, but the count says 5: **lost update**.

Count too high → consumer reads an **empty slot**. Count too low → producer **overwrites an unread slot**.

Fix: a counter shared by several writers needs `atomic_fetch_add()` or a mutex; several consumers need the read and the index update in one critical section.

Atomic operations (C11 <stdatomic.h>)

An atomic operation is **indivisible**: no other thread can see it half done. It can also **order** the ordinary memory accesses around it.

Operations

```c
#include <stdatomic.h>
_Atomic int count = 0;          /* or atomic_int */

/* load / store: never torn, never cached */
int v = atomic_load(&count);
atomic_store(&count, 0);

/* read-modify-write: ONE indivisible step */
atomic_fetch_add(&count, 1);    /* fixes H6 */
int old = atomic_exchange(&count, 5);

/* compare-and-swap: "if still e, write d" */
int e = atomic_load(&count);
while (!atomic_compare_exchange_weak(&count,
                                     &e, e * 2))
  ;  /* failed: e = current value, retry */

/* spinlock: a lock built from one atomic */
atomic_flag lk = ATOMIC_FLAG_INIT;
while (atomic_flag_test_and_set(&lk)) ;
/* ... critical section ... */
atomic_flag_clear(&lk);
```

Memory order

order	guarantees	typical use
`relaxed`	atomicity only, no ordering	counters, statistics
`acquire`	later accesses stay after this load	read a published index
`release`	earlier accesses stay before this store	publish an index
`acq_rel`	both, on a read-modify-write	lock-free updates
`seq_cst`	one global order for all (default)	when in doubt

Plain calls (`atomic_load`, `atomic_fetch_add` ...) use `seq_cst`; the `_explicit` versions take the order as a parameter.

x86: `lock xadd`, `lock cmpxchg` **ARM:** `ldaxr` / `stlxr`

What atomics do NOT do: two atomic operations in a row are not atomic together (Hazard 5 is still a race with atomic loads and stores). They **never block**: waiting means spinning, or a semaphore / mutex. Between processes they work only if lock-free: check `atomic_is_lock_free()` .

Solution 1 - Lock-free SPSC buffer (C11 atomics)

CODING EXAMPLE

```c
#include <stdatomic.h>

struct BufferData {
 _Atomic unsigned readIdx;  /* consumer writes */
 _Atomic unsigned writeIdx; /* producer writes */
 int buffer[BUFFER_SIZE];
};

/* ONE producer only. Returns 0 if full */
static int produce(struct BufferData *b, int item) {
 unsigned w = atomic_load_explicit(&b->writeIdx,
                 memory_order_relaxed);
 unsigned r = atomic_load_explicit(&b->readIdx,
                 memory_order_acquire);
 if (next(w) == r) return 0;       /* full */
 b->buffer[w] = item;              /* 1. write data */
 atomic_store_explicit(&b->writeIdx, next(w),
                 memory_order_release); /* 2. publish */
 return 1;
}
```

```c
/* Why every hazard is gone:                */
/* H1 nobody sleeps: no wakeup to lose      */
/* H2 data written before index published,  */
/*    data read before slot is freed        */
/* H3 release store / acquire load: writes  */
/*    before (2) are visible after it       */
/* H4 atomics are never torn or cached      */
/* H5 each index has exactly one writer     */
/* H6 no shared counter is needed           */

/* ONE consumer only. Returns 0 if empty */
static int consume(struct BufferData *b, int *item) {
 unsigned r = atomic_load_explicit(&b->readIdx,
                 memory_order_relaxed);
 unsigned w = atomic_load_explicit(&b->writeIdx,
                 memory_order_acquire);
 if (r == w) return 0;             /* empty */
 *item = b->buffer[r];             /* 1. read data */
 atomic_store_explicit(&b->readIdx, next(r),
                 memory_order_release); /* 2. free */
 return 1;
}
```

Circular buffer hazards: summary

Hazard	Kind	Symptom	Fix
H1 Lost wakeup	race condition	both sleep: deadlock	semaphore, or cond. var. under mutex
H2 Publish before write	race condition	read stale / empty slot	write data, then publish index
H2 Free before read	race condition	overwrite unread slot	read data, then advance readIdx
H3 Reordering, caching	data race	stale data, endless wait	release / acquire atomics, or locks
H4 Torn index update	data race	garbage index, lost update	one lock-free atomic per index
H5 Two producers	race condition	overwrite, lost item	mutex or semaphores around the write
H6 Two consumers	race condition	item consumed twice	mutex or semaphores around the read
H6 Shared counter ± 1	data race + race cond.	count ≠ contents	atomic_fetch_add or mutex

1 producer + 1 consumer: Solution 1, lock-free with C11 atomics (previous slide).

Many producers / consumers: Solution 2 semaphores, Solution 3 message queues, Solution 4 mutex + condition variables (next slides).

Semaphores

Linux semaphores are counting semaphores and are widely used to synchronize processes. When a semaphore has been created and an initial value assigned, two operations can be performed on it: **sem_wait()** and **sem_post()**.

There are two kinds of semaphores in Linux:

a. **named:** associated with a name (character string)

   created with:

```
sem_t *sem_open(const char *name, int oflag);
sem_t *sem_open(const char *name, int oflag,
                mode_t mode, unsigned int value);
```

a. **unnamed:**

   created with:

```
int sem_init(sem_t *sem, int pshared, unsigned int value);
```

Solution 2 - Semaphores: the idea

The following example is an implementation of the producer/consumer application where the producer and the consumers execute on different processes and use unnamed semaphores to manage the critical section and to handle synchronization.

In particular, the initial value of the semaphore (mutexSem) used to manage the critical section is set to one, thus ensuring that **only one process at a time can enter the critical section** by issuing first a sem_wait() and then a sem_post() operation.

```
int sem_wait(sem_t *sem);    /* wait   P proberen */
int sem_post(sem_t *sem);    /* signal V verhogen */
int sem_trywait(sem_t *sem);
```

The other two semaphores (dataAvailableSem and roomAvailableSem) will contain the current number of available data slots and free ones, respectively.

Solution 2 - Semaphores: producer and consumer

CODING EXAMPLE

```
struct BufferData {
 int readIdx;
 int writeIdx;
 int buffer[BUFFER_SIZE];
 sem_t mutexSem;
 sem_t dataAvailableSem;
 sem_t roomAvailableSem;
};

struct BufferData *sharedBuf;



/* Consumer routine */
static void consumer() {
 int item;
 while(1) {


/* Wait for availability of at least one data slot */
   sem_wait(&sharedBuf->dataAvailableSem);
/* Enter critical section */
   sem_wait(&sharedBuf->mutexSem);
/* Get data item */
   item = sharedBuf->buffer[sharedBuf->readIdx];
/* Update read index */
   sharedBuf->readIdx = (sharedBuf->readIdx + 1)%BUFFER_SIZE;
/* Signal that a new empty slot is available */
   sem_post(&sharedBuf->roomAvailableSem);
/* Exit critical section */
   sem_post(&sharedBuf->mutexSem);
/* Consume data item and take actions (e.g return)*/
  // ...
 }
}
```

CODING EXAMPLE

```
/* producer routine */
static void producer() {
 int item = 0;
 while(1)  {
/* Produce data item and take actions (e.g. return)*/
   // ...
/* Wait for availability of at least one empty slot */
   sem_wait(&sharedBuf->roomAvailableSem);
/* Enter critical section */
   sem_wait(&sharedBuf->mutexSem);
/* Write data item */
   sharedBuf->buffer[sharedBuf->writeIdx] = item;
/* Update write index */
   sharedBuf->writeIdx = (sharedBuf->writeIdx + 1)%BUFFER_SIZE;
/* Signal that a new data slot is available */
   sem_post(&sharedBuf->dataAvailableSem);
/* Exit critical section */
   sem_post(&sharedBuf->mutexSem);
 }
}
```

Solution 2 - Semaphores: main

CODING EXAMPLE

```
int main(int argc, char *args[])
{
 int memId;
 int i, nConsumers;
 pid_t pids[MAX_PROCESSES];
 if(argc != 2) {
   printf("Usage: prodcons <nConsumers>\n");
   exit(0);
 }
 sscanf(args[1], "%d", &nConsumers);

/* Shared memory, BEFORE fork():      */
/* every child must see the same buffer */
 memId = shmget(IPC_PRIVATE,
                sizeof(struct BufferData), 0666);
 sharedBuf = shmat(memId, NULL, 0);
 sharedBuf->readIdx = 0;
 sharedBuf->writeIdx = 0;

/* pshared = 1: shared between processes */
 sem_init(&sharedBuf->mutexSem, 1, 1);
 sem_init(&sharedBuf->dataAvailableSem, 1, 0);
 sem_init(&sharedBuf->roomAvailableSem, 1,
          BUFFER_SIZE);
```

CODING EXAMPLE

```
/* Launch producer process */
 pids[0] = fork();
 if(pids[0] == 0) {
/* Child process */
   producer();
   exit(0);
 }
/* Launch consumer processes */
 for(i = 0; i < nConsumers; i++) {
   pids[i+1] = fork();
   if(pids[i+1] == 0) {
     consumer();
     exit(0);
   }
 }
/* Wait process termination */
 for(i = 0; i <= nConsumers; i++) {
   waitpid(pids[i], NULL, 0);
 }
/* Release the shared memory */
 shmdt(sharedBuf);
 shmctl(memId, IPC_RMID, NULL);
 return 0;
}
```

Message queues

What is a Linux Message Queue?

A Linux Message Queue is a kernel-level Inter-Process Communication (IPC) mechanism that allows independent processes to exchange data by sending and receiving discrete, linked messages stored in kernel space.

Key Characteristics

- **Kernel-Managed Synchronization:** Much simpler than shared memory because there is no need to manage locks or semaphores manually. Everything is handled internally by the operating system kernel.
- **Producer-Consumer Model:** Operates as a stream of data flowing safely from producer process to consumer process.
- **System Calls:** Typically created and accessed using standard System V IPC system calls (`msgget`, `msgsnd`, `msgrcv`) or POSIX message queue APIs.

NOTE:

We will also see other means of synchronization that use message passing, such as sockets.

Solution 3 - Message queue: producer and consumer

CODING EXAMPLE

```
/* Message structure definition */
struct msgbuf {
 long mtype;
 int item;
};

int msgId;

/* Consumer routine */
static void consumer() {
 int retSize;
 struct msgbuf msg;
 int item;
 while(1)  {
/* Receive the message. msgrcv returns the size of the received message */
   retSize =  msgrcv(msgId, &msg, sizeof(int), PRODCONS_TYPE, 0);
   if(retSize == -1)  {  perror("error msgrcv"); exit(0); }
   item = msg.item;
/* Consume data item */
   // ...
 }
}
/* Producer routine */
static void producer()  {
 int item = 0;
 struct msgbuf msg;
 msg.mtype = PRODCONS_TYPE;
 while(1)  {
/* produce data item */
   // ...
   msg.item = item;
   msgsnd(msgId, &msg, sizeof(int), 0);
 }
}
```

Solution 3 - Message queue: main

CODING EXAMPLE CODING EXAMPLE

```
int main(int argc, char *args[]) {
 int i, nConsumers;
 pid_t pids[MAX_PROCESSES];
 if(argc != 2) {
   printf("Usage: prodcons <nConsumers>\n");
   exit(0);
 }
 sscanf(args[1], "%d", &nConsumers);
/* Initialize message queue */
 msgId = msgget(IPC_PRIVATE, 0666);
 if(msgId == -1) {
   perror("msgget");
   exit(0);
 }
/* Launch producer process */
 pids[0] = fork();
 if(pids[0] == 0) {
/* Child process */
   producer();
   exit(0);
 }
/* Launch consumer processes */
 for(i = 0; i < nConsumers; i++) {
   pids[i+1] = fork();
   if(pids[i+1] == 0) {
     consumer();
     exit(0);
   }
 }
/* Wait process termination */
 for(i = 0; i <= nConsumers; i++) {
   waitpid(pids[i], NULL, 0);
 }
/* Remove the queue: it outlives the processes */
 msgctl(msgId, IPC_RMID, NULL);
 return 0;
}
```

POSIX threads: the monitor pattern

Thread synchronization happens with mutexes and condition variables in a code pattern also called **Monitor**.

Monitors provide a mechanism for threads to temporarily give up exclusive access in order to wait for some condition to be met, before regaining exclusive access and resuming their task.

1) MUTEX LOCK

The operating system will put any thread trying to lock an already locked mutex in wait state, and such threads will be made ready as soon as the mutex is unlocked.

1) CONDITION WAIT

Condition variables can be used only by the methods of the monitor they belong to, and cannot be referenced in any way from outside the monitor boundary.

Solution 4 - Monitor: producer

CODING EXAMPLE

```
/* The mutex used to protect shared data */
pthread_mutex_t mutex;

/* Condition variables to signal availability of room and data in the buffer */
pthread_cond_t roomAvailable, dataAvailable;

#define BUFFER_SIZE 128
int buffer[BUFFER_SIZE];
int readIdx = 0;
int writeIdx = 0;

/* Producer code. Passed argument is not used */
static void *producer(void *arg) {
 int item = 0;
 while(1) {
/* Produce a new item and take actions (e.g. return) */
   //  ...
/* Enter critical section */
   pthread_mutex_lock(&mutex);
/* Wait for room availability */
   while((writeIdx + 1)%BUFFER_SIZE == readIdx)    {
     pthread_cond_wait(&roomAvailable, &mutex);
   }
/* At this point room is available Put the item in the buffer */
   buffer[writeIdx] = item;
   writeIdx = (writeIdx + 1)%BUFFER_SIZE;
/* Signal data availability */
   pthread_cond_signal(&dataAvailable);
/* Exit critical section */
   pthread_mutex_unlock(&mutex);
 }
 return NULL;
}
```

Solution 4 - Monitor: consumer and main

CODING EXAMPLE CODING EXAMPLE

```c
/* Consumer Code: the passed argument is not used */
static void *consumer(void *arg)
{
 int item;
 while(1)
 {
/* Enter critical section */
   pthread_mutex_lock(&mutex);
/* If the buffer is empty, wait for new data */
   while(readIdx == writeIdx)
   {
     pthread_cond_wait(&dataAvailable, &mutex);
   }
/* At this point data are available
  Get the item from the buffer */
   item = buffer[readIdx];
   readIdx = (readIdx + 1)%BUFFER_SIZE;
/* Signal availability of room in the buffer */
   pthread_cond_signal(&roomAvailable);
/* Exit critical section */
   pthread_mutex_unlock(&mutex);

/* Consume the item and take actions (e.g. return)*/
  // ...
 }
 return NULL;
}
```

```c
int main(int argc, char *args[])
{
 pthread_t threads[MAX_THREADS];
 int nConsumers;
 int i;
/* The number of consumers is passed as argument */
 if(argc != 2) {
   printf("Usage: prod_cons <numConsumers>\n");
   exit(0);
 }
 sscanf(args[1], "%d", &nConsumers);

/* Initialize mutex and condition variables */
 pthread_mutex_init(&mutex, NULL);
 pthread_cond_init(&dataAvailable, NULL);
 pthread_cond_init(&roomAvailable, NULL);

/* Create producer thread */
 pthread_create(&threads[0], NULL, producer, NULL);
/* Create consumer threads */
 for(i = 0; i < nConsumers; i++)
   pthread_create(&threads[i+1], NULL, consumer, NULL);

/* Wait termination of all threads */
 for(i = 0; i < nConsumers + 1; i++) {
   pthread_join(threads[i], NULL);
 }
 return 0;
}
```

Mutex, condition variable, semaphore

They solve different problems. A **monitor** is the shared data + one mutex + its condition variables, used together.

Mutex

a lock with an owner

`lock()` / `unlock()`

State: locked or unlocked, plus the owner thread

Only the owner may unlock it

Gives **mutual exclusion**: one thread at a time in the critical section

Cannot wait for a condition like "buffer not empty": you can only poll

Mutex + cond. variable

the monitor pattern

`wait()` / `signal()` / `broadcast()`

State: none, just a queue of sleeping threads

`wait()` unlocks the mutex and sleeps **in one atomic step**, then locks it again on wake-up

The condition lives in your shared data: **re-check it in a while loop**

A signal with nobody waiting is lost

Semaphore

a counter of permits

`sem_wait (P)` / `sem_post (V)`

State: an integer ≥ 0

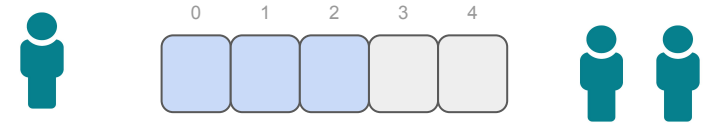
`sem_wait` takes a permit or blocks; `sem_post` adds one

No owner: any thread or process can post

Remembers posts: no lost wakeup. Counts resources (items, free slots)

Mutex: who may touch the data. **Condition variable:** when it is worth touching it. **Semaphore:** how many may go ahead.

Analogy: the bakery shelf



baker shelf with N spots **customers**

Baker = producer, customers = consumers, shelf = ring buffer. Rule: nobody may touch the shelf while someone else is using it.

Mutex

the one pair of tongs

Only the person holding the tongs may touch the shelf, and the one who picked them up must put them back. A customer who finds the shelf empty can only put the tongs down and come back later, again and again: **polling**.

Condition variable

the bench and the bell

A customer who finds the shelf empty puts the tongs down **and** sits on the bench in one move (`wait`). The baker adds bread and rings the bell (`signal`). The customer takes the tongs and **looks again**: another customer may have been faster (`while`). A bell rung to an empty bench is heard by nobody.

Semaphore

the board with two numbers

Next to the shelf a board shows **Loaves: 3** and **Free spots: 2**. `sem_wait`: if the number is > 0, take 1 off and go; if it is 0, queue in front of the board. `sem_post`: add 1, and the first one in the queue goes.

Customer: wait(Loaves), take bread, post(Free spots). Baker: wait(Free spots), add bread, post(Loaves). **The number stays written** (no lost wakeup) and **anyone can change it** (no owner).

Same consumer, three ways

The consumer of the ring buffer must wait until it is **not empty**.

Mutex only

```
pthread_mutex_lock(&mutex);
while (readIdx == writeIdx) {
  pthread_mutex_unlock(&mutex);
  usleep(100);   /* poll */
  pthread_mutex_lock(&mutex);
}
item = buffer[readIdx];
readIdx = next(readIdx);
pthread_mutex_unlock(&mutex);
```

Correct but **burns CPU** and adds up to one poll period of latency.

Mutex + condition variable

```
pthread_mutex_lock(&mutex);
while (readIdx == writeIdx)
  pthread_cond_wait(&dataAvailable,
                    &mutex);
item = buffer[readIdx];
readIdx = next(readIdx);
pthread_cond_signal(&roomAvailable);
pthread_mutex_unlock(&mutex);
```

Sleeps without CPU. The condition can be **any expression** on the shared data.

Semaphores

```
sem_wait(&dataAvailableSem);
sem_wait(&mutexSem);
item = buffer[readIdx];
readIdx = next(readIdx);
sem_post(&mutexSem);
sem_post(&roomAvailableSem);
```

The **count is the condition**. Works between processes in shared memory (`pshared = 1`).

In the analogy: column 1 walks back to the shelf every 100 µs, column 2 sits on the bench until the bell rings, column 3 waits at the loaf dispenser.

Comparison table

	Mutex	Mutex + cond. var.	Semaphore
Has memory	locked / unlocked	no, the state is your data	yes, a counter
Owner	yes, only it unlocks	the mutex has one	no, anyone posts
Waits for	the lock	any condition	count > 0
Lost wakeup	—	yes, if the state changes outside the mutex	no
Priority inheritance	`yes (PTHREAD_PRIO_INHERIT)`	through its mutex	no
Between processes	`PTHREAD_PROCESS_SHARED`	`PTHREAD_PROCESS_SHARED`	`pshared = 1, or sem_open()`
In a signal handler	no	no	`sem_post() is safe`
Best for	short critical sections	complex conditions, broadcast	counting resources, signalling

Real-time note: a semaphore used as a lock has no owner, so the kernel cannot apply priority inheritance: a low-priority holder can block a high-priority task indefinitely (**priority inversion**).

Common pitfalls and how to choose

Condition variable

`if` instead of `while` around `wait()`: a spurious wake-up, or another consumer that took the item first, makes you read an **empty slot**.

Changing the shared state **without** holding the mutex, then signalling: the waiter can test, miss the signal, and sleep forever (**lost wakeup**).

Semaphore

Used as a lock, a stray extra `sem_post` lets **two threads** into the critical section: there is no owner to check.

Wrong order `sem_wait(&mutexSem)` before `sem_wait(&dataAvailableSem)`: sleeping while holding the lock = **deadlock**.

Mutex

Unlocking from another thread is undefined; locking twice from the same thread **deadlocks** (unless recursive).

Holding it while sleeping, doing I/O or waiting for another lock: long blocking, and **deadlock** if locks are taken in different orders.

How to choose

Protect shared data?
→ mutex

Wait for a state of that data (not empty, all done)?
→ mutex + condition variable (Solution 4)

Count resources or signal between processes?
→ semaphores (Solution 2)

Just move data between processes?
→ message queue (Solution 3)

1 producer + 1 consumer, no blocking?
→ lock-free ring buffer (Solution 1)